

Pure-Gemma Evolution Matrix: Agentic Workflows for Python Test Generation

Johnprice Osagie

Politecnico di Torino
s344613@studenti.polito.it

Matteo Gastaldello

Politecnico di Torino
s345026@studenti.polito.it

Deniz Eren Gencay

Politecnico di Torino
s339282@studenti.polito.it

Erestina Vreto

Politecnico di Torino
s345027@studenti.polito.it

Abstract

This report studies how lightweight LLM agent workflows affect automatic unit test generation in Project A1. We compare 8 models from the Gemma family (Generation 3 vs Generation 4), 11 orchestration patterns routed via Lang-Graph, and up to 4 prompting styles on a 25-task EvalPlus subset. To ensure robustness, the execution pipeline incorporates a custom parser and isolated execution environments.

The best configurations generally pair the largest models with workflows and prompts that match their capabilities. However, the broader picture reveals significant trade-offs. We observed generational breakthroughs for smaller models, alongside structural collapses and an efficiency trap for medium-density models, where newer generations underperform compared to their predecessors. Baseline runs remain strong competitors, and few-shot prompting proves to be the most reliable style on average. More complex workflows often impose a high token cost—an "Efficiency Collapse" exceeding 85% in Zero-Shot mode—for relatively modest gains. Ultimately, multi-agent methods do not consistently dominate; rather, they provide substantial help in targeted settings when the architecture, prompt style, and model family are well aligned.

1 Introduction

Automatic test generation serves as an excellent setting for evaluating LLM-based software engineering systems. A high-quality generated test suite must do more than produce syntactically valid Python: it needs to exercise the intended behavior of the target function, cover edge cases, and remain readable enough for developers to inspect or reuse.

This project evaluates whether lightweight agent workflows can improve this process on the EvalPlus benchmark (Liu et al., 2023). The codebase implements several orchestration patterns where one or more LLM calls generate, critique, merge, or refine

candidate pytest suites. The goal of this report is to summarize the impact of these workflows, staying faithful to the metrics and implementation used in the repository.

1.1 Research Questions

We focus on three primary questions:

- **RQ1:** Which model, workflow, and prompt combinations achieve the strongest results?
- **RQ2:** Do lightweight multi-step workflows consistently outperform the direct baseline?
- **RQ3:** How do performance, token usage, and auxiliary quality metrics balance against each other?

2 Repository Scope

The experiments utilize the EvalPlus subset stored in `data/evalplus_subset` (Liu et al., 2023), which contains 25 problems. The aggregated analysis covers 265 model-agent-prompt combinations collected across multiple experiment sweeps.

The grid is not perfectly uniform across all models. We evaluated Gemma-1B, 4B, 12B, 27B, and Gemma-31B on the full 11-agent by 4-prompt grid, while Gemma-4-2B, 4-4B, and 4-26B were evaluated on a narrower subset of workflows and prompt styles. Even with this unbalanced grid across model families, the comparisons provide a highly informative look at the performance landscape.

3 Methodology

3.1 Models and Workflows

We evaluate 8 Gemma variants from the Gemma 3 family (Gemma Team, 2025):

- **Generation 3:** Gemma-1B, 4B, 12B, and 27B.

- **Generation 4:** Gemma-4-2B, 4-4B, 4-26B, and Gemma-31B.

The tested workflows include *Baseline* (Brown et al., 2020), *Actor-Critic* (Le et al., 2022), *Adversarial* (Zhang et al., 2026), *Competitive* (Du et al., 2023), *Hybrid* (Li et al., 2024), *CoA* (Zhang et al., 2024), *SOA* (Wang et al., 2024), *Swarm* (Wang et al., 2025), *Consensus* (Wang et al., 2022), *Self-Healing* (Charalambous et al., 2023), and *Atomic Swarm* (Chen et al., 2023). Drawing inspiration from recent LLM literature, we adapt these concepts specifically for collaborative test generation. **LangGraph** handles the orchestration, providing stateful multi-agent workflows and complex logic routing. Atomic Swarm adapts AgentVerse-style multi-agent collaboration (Chen et al., 2023), while Consensus relies on a self-consistency-inspired voting scheme (Wang et al., 2022). Depending on the workflow, the system might generate multiple candidates for merging, add moderation steps, or split prompts into smaller testing subgoals.

3.2 Execution Setup and Custom Parser

Before execution, LLM outputs pass through a custom regex-based parser. This parser performs **Markdown Sanitization** to extract code from triple backtick blocks, **Conversational Filtering** to remove introductory phrases that cause syntax errors, and **Import Normalization** to strip hallucinated modules. By validating the code with Python’s `ast` module, we ensure structural integrity and prevent broken scripts from reaching the execution environment.

The experimental runner then loads a problem prompt, invokes the selected workflow, and evaluates the resulting suite against the canonical reference implementation. Custom instrumentation with `sys.settrace` provides fine-grained tracking, while multi-process isolation paired with hardware kill-switches ensures safety and deterministic evaluation. The backend relies on the Google GenAI client, implementing repository-side rate limiting and token counting.

3.3 Metrics

In this codebase, **Pass Rate** tracks the average fraction of passing pytest checks against the canonical reference solution. It measures how often the generated tests are valid with respect to the reference implementation.

We also report:

- **Line coverage and branch coverage:** measured via `pytest-cov` (pytest-cov contributors, 2026).
- **Mutation score:** measured via `mutmut` (boxed, 2026) or a regex-based fallback.
- **Maintainability and diversity metrics:** computed from the generated code.
- **Token efficiency:** Pass Rate per 1,000 total tokens.

It is worth noting that the evaluator collects coverage over a temporary directory rather than restricting it strictly to `solution.py`. Therefore, these values should be read as relative comparisons inside this project, not as strict source-only coverage benchmarks.

4 Results

4.1 Best Configurations by Model

The strongest single configuration in our runs is **Gemma-31B + Hybrid + zero-shot**, achieving a Pass Rate of **0.976**. Other top configurations include **Gemma-4-26B + Baseline + few-shot** (0.905), **Gemma-27B + Hybrid + SCoT** (0.807), **Gemma-4B + Baseline + few-shot** (0.770), and **Gemma-12B + Actor-Critic + CoT** (0.717).

Two patterns emerge from this data. First, scaling the model size reliably improves peak scores. Second, the winning workflow varies significantly by model: while large models often benefit from merge-or-refine patterns like Hybrid, smaller models generally achieve their peak Pass Rate using the plain baseline. Smaller models seem to lack the capacity to follow intricate multi-agent instructions, causing complex workflows to drag down their overall accuracy.

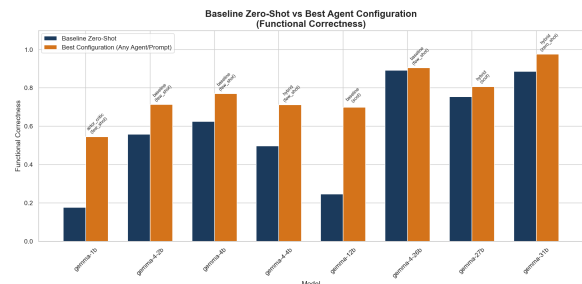


Figure 1: Baseline Pass Rate versus Best Configuration across model sizes.

Model	Best Setup	Pass Rate	Pass Rate / 1k Tok.
Gemma-31B	Hybrid + Zero-Shot	0.976	0.116
Gemma-4-26B	Baseline + Few-Shot	0.905	0.096
Gemma-27B	Hybrid + SCoT	0.807	0.269
Gemma-4B	Baseline + Few-Shot	0.770	0.475
Gemma-12B	Actor-Critic + CoT	0.717	0.249
Gemma-4-2B	Baseline + Few-Shot	0.713	0.227
Gemma-4-4B	Hybrid + Few-Shot	0.713	0.139
Gemma-1B	Actor-Critic + Few-Shot	0.546	0.103

Table 1: Best Pass Rate configurations in the repository aggregate.

4.2 Do Agent Workflows Win?

While multi-step workflows can be highly effective, they do not consistently dominate the baseline across the full aggregate.

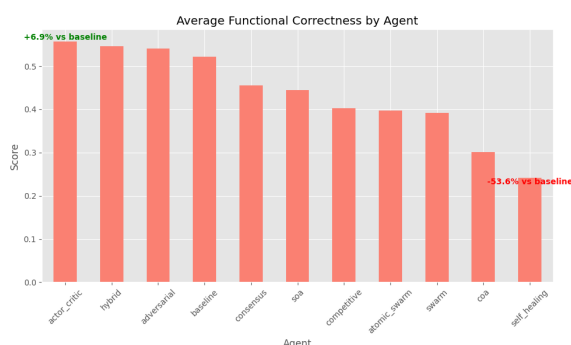


Figure 2: Average performance by agent architecture.

The **Baseline** actually achieves the strongest average Pass Rate across all rows, followed by **Hybrid**, **Adversarial**, and **Actor-Critic**. This demonstrates that adding more agent structure does not guarantee better outcomes.

On Gemma-31B and Gemma-27B, merge-and-refine workflows like Hybrid reach best-in-class performance. Conversely, on Gemma-4-26B, Gemma-4B, and Gemma-4-2B, a straightforward baseline prompt wins. Architecture is most helpful when it aligns with the model’s inherent capacity; otherwise, orchestration simply adds noise, computational cost, and potential failure points.

4.3 Previous vs. Current Generation (Gen 3 vs Gen 4)

The introduction of the Gemma 4 family reveals interesting generational trade-offs:

- **The Breakthrough (2B vs 1B):** The raw, zero-shot capabilities of the 2B model completely surpass the most heavily agent-assisted peak of the 1B version, resulting in a massive **+366%** performance jump.

- **The Disappointment and Efficiency Trap (4B vs 4B):** Unexpectedly, the new 4B model suffers a global performance drop of **-27%**. These high-density models exhibit prompt fragility or "structural collapse." To compensate, the Gen 4 4B model is forced to rely on token-expensive multi-agent pipelines just to catch up to the standard baseline of the older 4B model.
- **The Efficiency Gap (26B vs 27B):** The 26B model achieves an impressive 89% zero-shot accuracy, effectively rendering complex multi-agent architectures unnecessary. However, Gen 4 consumes **~11x more tokens** due to its reasoning overhead. Ultimately, Gen 4 models offer higher intelligence but prove to be more capricious and expensive.

4.4 Worst Configurations and Failure Modes

Agentic architectures carry specific and recognizable failure modes:

- **Efficiency Collapse:** Running Consensus or Swarm agents in Zero-Shot mode causes a dramatic efficiency drop exceeding **85%**.
- **Correctness Degradation:** Deploying high-overhead architectures on 1B models without Few-Shot support reduces functional correctness by up to **65%**.
- **Maintainability Pitfalls:** The over-engineered logic that results from combining Swarm and CoT reduces the Maintainability Index by up to **37%**.
- **Architecture Overkill:** A "Complexity Penalty" triggers whenever the multi-call overhead exceeds the model’s natural instruction-following capacity.

4.5 Prompting Effects

Prompt style dictates performance almost as much as workflow choice. **Few-shot prompting** delivers the highest average Pass Rate, followed by zero-shot, SCoT, and CoT. This suggests that standard CoT often overcomplicates the testing task. The optimal style shifts dynamically by scale: **4B** relies heavily on Few-Shot, **27B** and **4-31B** gain a substantial boost from CoT, and **12B** frequently generates verbose, inaccurate responses when subjected to CoT.

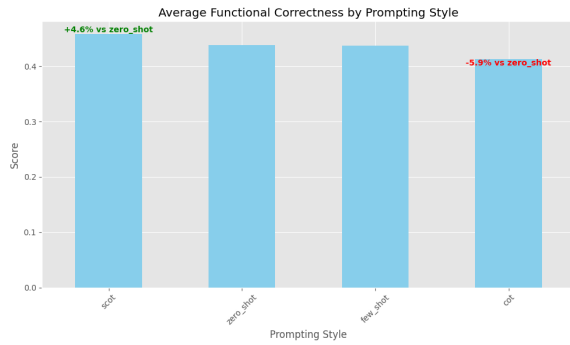


Figure 3: Average performance comparison across Zero-shot, Few-shot, and SCoT prompting styles.

Gemma-4-4B provides a clear warning regarding prompt sensitivity. Its baseline Pass Rate drops from **0.629** with few-shot to **0.169** with SCoT—a 73% decrease. Newer chat-oriented models show high sensitivity to rigid templates, meaning structured reasoning prompts should not be treated as universal defaults.

4.6 Efficiency and Trade-offs

When we factor in token efficiency, the ranking changes substantially.

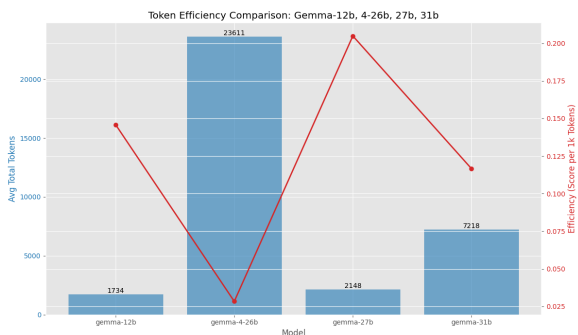


Figure 4: Token efficiency versus token usage across Gemma configurations.

Gemma-31B hits peak Pass Rate, but it requires a massive token budget to do so. Gemma-27B offers a much stronger compromise, reaching **0.807** with far better efficiency than Gemma-4-26B. On the lower end, the Gemma-4B baseline few-shot setup acts as a highly efficiency-oriented configuration, achieving **0.475 Pass Rate per 1k tokens**.

The "agentic tax" is clearly visible in these results: extra critique, merge, or coordination steps deliver only marginal robustness gains while multiplying token consumption. This trade-off is critical when determining the feasibility of continuous testing pipelines.

4.7 Quality Metrics Beyond Pass Rate

The auxiliary metrics provide valuable structural context. Richer test suites generally cover more code, while cleaner suites achieve higher Pass Rate and maintainability. Generation quality, code coverage, similarity, and bloat all interact in complex ways.

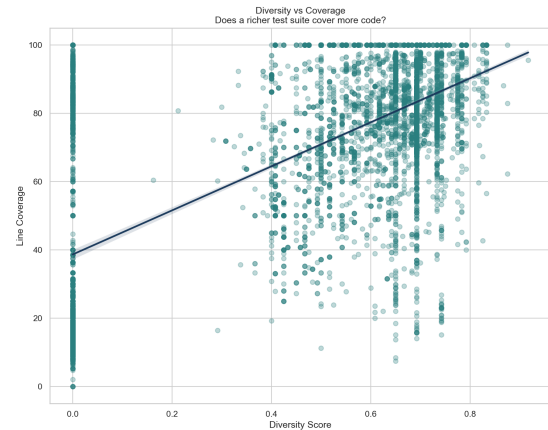


Figure 5: Diversity score versus line coverage. Richer suites generally cover more code, but the spread shows that diversity alone is not sufficient.

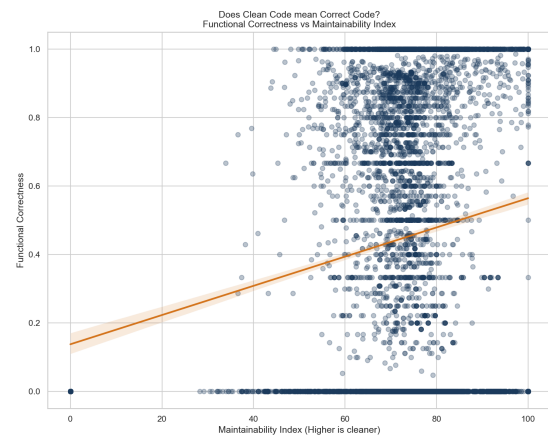


Figure 6: Pass Rate versus maintainability index. Cleaner suites often achieve higher Pass Rate, but the correlation remains noisy.

We analyzed the correlation matrix across these dimensions:

- **Coverage vs. Correctness** correlates strongly (≈ 0.72); higher coverage generally predicts higher correctness.
- **Coverage vs. Diversity** correlates moderately (≈ 0.65).
- **Coverage vs. Mutation Score** correlates surprisingly weakly (≈ 0.31). This highlights

that while coverage measures breadth, mutation measures depth.

These correlations expose a notable **Logic Gap**: high coverage does not guarantee functional correctness. It is entirely possible for buggy code to hit high coverage metrics. For instance, the Atomic Swarm architecture reaches $\sim 100\%$ coverage on buggy code simply by writing circular tests that assert the generated bug.

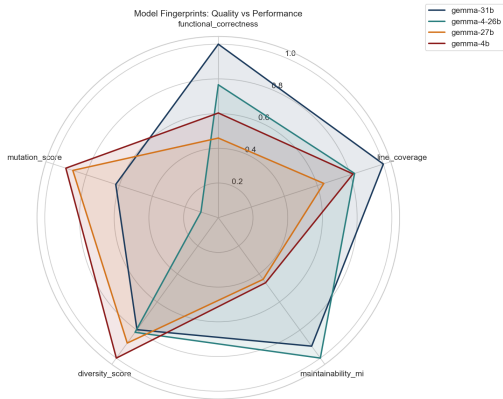


Figure 7: Model fingerprints across performance and quality-related metrics.

The radar view in Figure 7 confirms that no single model dominates all dimensions. The Gemma-31B best-configuration misses the top spot for coverage and mutation score—the highest coverage actually comes from *SOA + few-shot*. Lower similarity and lower bloat accompany diverse suites, but they do not guarantee higher Pass Rate. Ultimately, the metrics align only partially.

Because of this, deployment decisions must rest on specific priorities: maximum test validity, broad exploratory coverage, or minimal token cost.

5 Limitations

We constrain this report to what the repository directly measures:

- The experiments use a 25-problem EvalPlus subset rather than the full benchmark.
- The model-agent-prompt grid lacks complete uniformity across model families.
- Pass Rate measures generated-test validity against canonical solutions.

- Coverage reflects the temporary workspace, requiring comparative rather than absolute interpretation.
- The tested workflows are simplified orchestration patterns, serving as shorthand rather than claims of full autonomy.

6 Conclusion

Lightweight agent workflows can improve automatic unit test generation when large models are paired with aligned prompts. Although Gemma-31B with the Hybrid workflow wins the aggregate, the baseline remains a remarkably strong competitor and the top average workflow overall.

Larger models raise the performance ceiling, and few-shot prompting provides the safest default. However, rigid structured prompts can derail newer models, and extra orchestration must always justify its token cost. The next project phase moves away from parameter scaling to target long-context handling for software engineering tasks, evaluating Recursive Language Models (Zhang et al., 2025) and JEPA-style architectures (Huang et al., 2025).

7 Future Work

The 2026 baseline achieved high efficiency: we tested up to 31B parameters while keeping total API costs under 4 euros. Future iterations will build on more efficient model families to improve benchmarks without inflating costs proportionally.

In 2027, the primary technical bottleneck shifts to long context. As medium-size models solve existing software engineering benchmarks, further progress will depend on sustaining reasoning over long inputs, large codebases, and complex workflow states. Recursive Language Models (Zhang et al., 2025) and JEPA-inspired architectures (Huang et al., 2025) tackle the long-context barrier directly, potentially enabling multi-step workflows that go far beyond standard prompt-and-merge patterns.

References

- boxed. 2026. [mutmut: python mutation tester](#).
- Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.

- Yiannis Charalambous and 1 others. 2023. A new era in software security: Towards self-healing software via large language models and formal verification. *arXiv preprint arXiv:2305.14752*.
- Weize Chen and 1 others. 2023. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*.
- Yilun Du and 1 others. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*.
- Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.
- Hai Huang, Yann LeCun, and Randall Balestriero. 2025. Llm-jepa: Large language models meet joint embedding predictive architectures. *arXiv preprint arXiv:2509.14252*.
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven CH Hoi. 2022. Coderl: Collaborative training of decoding and rescoring for software-program synthesis. *arXiv preprint arXiv:2207.01780*.
- Junyou Li and 1 others. 2024. More agents is all you need. *arXiv preprint arXiv:2402.05120*.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*.
- pytest-cov contributors. 2026. [pytest-cov: Pytest coverage plugin](#).
- Lijie Wang and 1 others. 2024. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Dai. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Zhen Wang and 1 others. 2025. Swarms of large language model agents. *arXiv preprint arXiv:2502.13146*.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. 2025. Recursive language models. *arXiv preprint arXiv:2512.24601*.
- Xin Zhang and 1 others. 2026. Test vs mutant: Adversarial llm agents for robust unit test generation. *arXiv preprint arXiv:2602.08146*.
- Yusen Zhang and 1 others. 2024. Chain of agents: Large language models collaborating on long-context tasks. *arXiv preprint arXiv:2406.02818*.